VITYARTHI BYOP FUNDAMENTAL IN AI-ML

SUBMITTED BY: VEDANT CHOUKSEY
25BCE10910

COURSE: FUNDAMENTALS IN AIML
FACULTY: M.K. JAYANTHI

EMOTION DETECTOR CLI (AIML) PROJECT

INTRODUCTION

In the modern digital age, most forms of communication are done through text, whether it is through chat messages, emails, social media, reviews, or online feedback. When people write something, they generally want to express their emotions, which could be happiness, sadness, anger, fear, love, or surprise. Generally, every person is able to understand the emotions behind the text, but for the computer, understanding the emotions behind the text is not so simple.

The project developed is called Emotion Detector CLI App, which is a simple AI/ML-based project that tries to find out the emotion behind the text. The user is required to input the text, and the system tries to find out the most suitable emotion using the machine learning algorithm. This project is developed using the Python programming language.

The main idea behind this project is to learn how text data is processed and converted into features and then classified using machine learning algorithms. This is a simple project that is helpful in gaining hands-on experience in AI and ML.

MOTIVATION OF THE PROJECT

The primary reason for undertaking this project was inspired by the increasing tendency of people to use text-based communication. Today, people are more likely to express their emotions online than ever before, whether it is through a chat, a post, a comment, or a review. If a computer system can identify the emotion behind this text, then it can become even more interactive.

Another reason for undertaking this project was that I wanted to learn how Artificial Intelligence and Machine Learning can be applied to real-life problems. Instead of undertaking a project that only focuses on theory, I wanted to undertake a project that was practical, easy to understand, and useful for learning concepts like text preprocessing, feature extraction, model building, and prediction.

This project was also a source of motivation for me because emotion detection is a very interesting domain of Natural Language Processing, and there are a number of applications of this domain.

Problem statement

Human beings can easily comprehend the emotional content of the sentence. However, it is impossible for the machine to do it on its own. The machine can only see the raw words unless it is programmed to do otherwise.

The main problem addressed in this project is:

To design a system that can automatically recognize the emotion of the given text input by the user with the help of machine learning algorithms.

The system should be able to:

- Take the text input from the user,
- Process the text correctly,
- Classify the text into one of the predefined emotions,
- Display the prediction result,
- Keep the prediction history.

OBJECTIVE OF THE THE PROJECT

The main objectives of this project are as follows:

- To implement a simple emotion detection system by using machine learning.
- To implement a text classification system for certain emotions such as joy, sadness, anger, fear, love, and surprise.
- To understand how text preprocessing is done in Natural Language Processing.
- To utilize TF-IDF Vectorization for text conversion.
- To implement a classification system by using Logistic Regression.
- To implement a user-friendly command-line-based application for emotion prediction.
- To store the history of predictions.

This project will help bridge the gap between learning and practice.

EXISTING METHODS

There are already a number of ways that can be used for emotion detection and sentiment analysis of a particular text. Some of the common methods are as follows:

1. Rule-Based Methods

This type of method works on the basis of a number of predefined rules and keywords for emotion detection. For example, if a sentence contains words like “happy” or “excited,” then it may be considered a joyful emotion.

2. Lexicon-Based Methods

This type of method works on the basis of a dictionary or a lexicon, where all the words are already associated with a particular emotion. The system then checks the words in the sentence and then decides the emotion.

3. Traditional Machine Learning Methods

This type of method works on the basis of a number of machine learning algorithms, such as:

- Naive Bayes
- Logistic Regression
- Support Vector Machine (SVM)
- Decision Trees

This type of machine learning algorithm can be combined with a number of feature extraction algorithms, such as:

- Bag of Words
- TF-IDF

4. Deep Learning Methods

This type of machine learning algorithm is more complex and can understand the context of a particular sentence, especially if a large dataset is available. Deep learning algorithms, such as:

- RNN
- LSTM
- GRU
- BERT/Transformer

PROS AND CONS OF THE STATED METHODS

A. Rule-Based Methods

Pros:

- Very easy to build
- Very fast and simple
- No need to undergo training

Cons:

- Can't properly understand sentence context
- Only suitable for limited cases
- Not suitable for real-life text data

B. Lexicon-Based Methods

Pros:

- Very easy to use
- No need to undergo training
- Suitable for basic emotion analysis

Cons:

- Only suitable for words available in the dictionary
- Can't handle cases of sarcasm and hidden meanings
- Not suitable for natural conversations

C. Traditional Machine Learning Methods

Pros:

- Balances simplicity and performance well
- Suitable for small and medium-sized datasets
- Easier to undergo training and implementation

Cons:

- Requires preprocessing and feature extraction
- Performance depends on the quality of the dataset
- Not suitable for very complex texts

D. Deep Learning Methods

Pros:

- Better understanding of context

- Higher accuracy with larger datasets
- Suitable for more complex applications

Cons:

- Requires more data and computing power
- More complex to undergo implementation
- Not suitable for small beginner projects

HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements

The project does not require very high-end hardware. It can run on a normal laptop or desktop system.

- Processor: Intel i3 / i5 or above
- RAM: 4 GB or more
- Storage: At least 500 MB free space
- Keyboard and Monitor
- Internet connection (for installing libraries if needed)

Software Requirements

The software used in this project includes:

- Operating System: Windows / Linux / macOS
- Programming Language: Python 3.x
- IDE / Editor: VS Code / PyCharm / IDLE
- Libraries Used:

    1. pandas
    2. scikit-learn
    3. pickle
    4. re
    5. os
    6. datetime

METHODOLOGY AND GOAL

The project was developed in a step-by-step manner so that the emotion detection process becomes simple and understandable.

Methodology Used
The functionality of the system can be summarized as follows:

- Dataset collection: Collection of dataset through creation of a CSV file, where each individual line in the dataset contains information concerning a specific emotion.

- Cleaning text: Text was processed to remove unnecessary characters and other symbols, converted to lower case, and any extra spaces were removed.

- Data separation: Data was split into two operational areas called training and test sets.

- Conversion of textual data to feature sets: As the textual databases of data used must be converted into numerical form for use with the machine learning model, TF-IDF vectorization was used to accomplish this.

- Model training: The textual data was the input to the model used for model training, which was a logistic regression algorithm.

- Model testing: The performance of the model was evaluated through testing.

- Model saving: The model is saved using the Pickle module for future reference.

- Building a Command Line Interface application: Development of a Command Line Interface Application to create a way to interact with users.

Project objectives:

The main goal of this project is to produce a simple yet functional AI-based system that is capable of analysing the emotion of the corresponding textual input provided by the user.

FUNCTIONAL MODULES DESIGN AND ANALYSIS

The system that comprises the project's structure is made up of several modules, each of which is responsible for a different function for the system as a whole.

Module Description:

1. Dataset Module

This is where the labeled text data is located for training purposes.

2. Text Preprocessing Module

This is where input data is cleaned and normalized be for actual training / prediction can happen.

3. Training Module

Machine Learning Model will be trained on the cleaned and normalized Dataset.

4. Prediction Module

New data from users will be inputted and the predicted emotion will be outputted.

5. Confidence Score Module

Once the prediction is complete, this module will give an output of how confident the model was that the weed was accurately predicted based on user input data.

6. History Module

This module will save all previous emotion generations from the Past to a text file.

7. User Interaction Module

This module contains the command line user interface (CLI) menu for the user to interact and use the application.

By designing the project in this manner, by module segmentation, we can create a clearer understanding of how the system works, how to test each of the modules, and how to improve the modules over time.

ALGORITHM DEVELOPMENT

The main algorithm used in this project is Logistic Regression.

This algorithm is widely used for classification tasks and works well for text classification when combined with TF-IDF features.

Steps of the Algorithm

1. Read the dataset from dataset.csv
2. Clean the text data
3. Split the dataset into training and testing sets
4. Convert text into TF-IDF vectors
5. Train the Logistic Regression model
6. Predict emotions for test data
7. Evaluate the model performance
8. Save the trained model
9. Load the saved model in the main application
10. Predict emotion for user-entered text

This algorithm was chosen because it is simple, efficient, and suitable for beginner-level machine learning projects.

SOFTWARE ARCHITECTURAL DIAGRAM

The working flow of the project can be shown as follows:

User Input Text
↓
Text Cleaning / Preprocessing
↓
TF-IDF Feature Extraction
↓
Machine Learning Model
(Logistic Regression)
↓
Emotion Prediction
↓
Confidence Score Display
↓
Save in History File

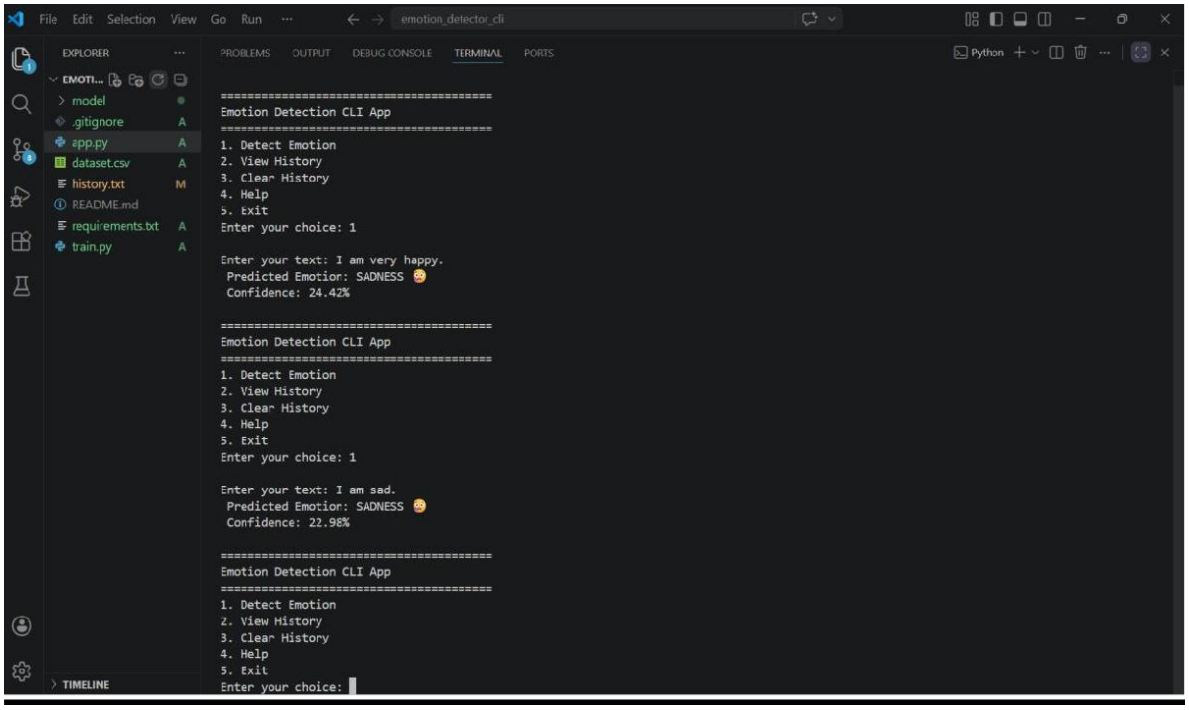
This architecture shows how the user input is processed step by step until the final prediction is generated.

CODING

The project is comprised of two main components, with the first part (app.py) being an application that runs all of the necessary code to read/write/read/write/learn the model used in our second component. The second component (train.py) is an application that takes input from users, predicts their emotional state based on their response, and displays this information, as well as managing the command line.

Code has been written so that it is easy to follow and make changes to later if necessary, both for any future enhancements needed by the Developer(s) and/or others working on the project.

OUTPUT



KEY IMPLEMENTATIONS OUTLINE OF THE SYSTEM

The main implementations completed within the scope of the project include:

- Using Regular Expressions to clean text input
- Using train_test_split() to split the dataset
- Using TF-IDF for feature extraction
- Training a Logistic Regression model
- Calculating prediction confidence with predict_proba()
- Saving the model with Pickle
- Keeping a record of prediction history in a text file
- Creating a menu-driven command-line interface

The above implementations comprise the primary functionality of the system.

SIGNIFICANT PROJECT OUTCOMES

The project was an excellent example of using machine learning to classify emotions based on textual input.
The many results of the project include:

- A functional AI/ML emotion classification robot that can identify an emotion from text instantly (in real-time).

- An example of demonstrating the application of machine learning and NLP principles.

- It's superbly straightforward, yet it successfully implements text classification.

- It's a clean and simple CLI-based interface that allows for easy testing and use.

Overall, the project provided new insights on emotion recognition theory as well as practical application of emotion recognition.

TESTING AND REFINEMENT

Following the construction of the emotion-based model, it was validated using several sentences corresponding to each of the emotions in order to ascertain the model's proper detection of the intended class by this system. The methods used to evaluate the model include:

- Accuracy Score
- Classification Report
- Confusion Matrix

The following changes occurred during development to improve performance and reliability:

Improved text cleaning provided more consistent input
using stratify=y during the train/test split balanced all class distributions
added bigram feature captures during TF-IDF processing
increased iterations of Logistic Regression to provide a more stable training process

Tests conducted throughout the refinement phase increased the model's effectiveness and reliability as well.

PROJECT APPLICABILITY ON REAL-WORLD APPLICATIONS

Detecting emotion has many practical uses in everyday life. Examples of how these types of technologies can be effectively used include;

- Chatbots and virtual assistants
- Customer review & feedback analysis

- Mental health support systems
- Social media monitoring
- Online learning platforms
- Call center and support systems
- HCI (human-computer interaction)

By using this type of technology, organizations and apps can gain an enhanced understanding of what users are feeling and provide them with a better response.

CONTRIBUTION / FINDINGS OF THE PROJECT

This project has significantly advanced our knowledge of the practical application of detecting emotions from text.
Key observations include:

- A very simple ML model can accurately classify emotion.
- The results of a text classification model can benefit significantly from having cleanly preprocessed input text.
- TF-IDF is ideal for small text classification problems.
- Logistic Regression is a good starting point for new projects in NLP and good for practical use.
- Creating a command-line interface (CLI) will enhance the usability and interactivity of this project.

This project has also provided insight into how to use Artificial Intelligence productively and in an easy to comprehend manner.

LIMITATION / CONSTRAINTS OF THE SYSTEM

However, the entire experiment is still limited in several ways:

1) The Data Set is Very Small
2) Limited Number of Emotions Can be Recognised
3) The Model May Not Recognise Sarcasm and Indirect Emotions
4) Predictions May Be Poor Due to the Quality of the Training Data
5) System May Not Handle Longer and More Complex Sentences Well
6) The Command Line Interface is Also Very Basic Compared to GUIs and Web Applications

These limitations are to be expected for a beginner project, and will be worked on for future versions.

CONCLUSIONS

The Emotion Detector CLI App is an AI/ML-based project that highlights the use of machine learning for emotional detection from inputted text. This project demonstrates how text can be processed, features can be extracted, classified, and communicated to users in a usable manner; therefore, demonstrating how machine learning models are developed and utilized for predictive modeling in real life.

Overall, this project provides an excellent example of utilizing AI to address a relevant problem in a very easy manner.

FUTURE ENHANCEMENTS

There are several ways this project could be further improved. Some ideas for future improvement include:

- Expanding the dataset size and creating a more realistic dataset
- Adding additional emotion categories beyond the current list
- Enhancing the preprocessing of text through the use of libraries like NLTK or spaCy
- Creating a GUI application using Tkinter
- Creating a web application using a Framework such as Flask or Streamlit
- Adding voice input functionality
- Building additional advanced deep learning models (i.e. LSTM, GRU, BERT)
- Adding support for multiple languages

Making these improvements would result in a more accurate, advanced, and user friendly version of the application.

REFERENCES

- Scikit-learn Documentation
  https://scikit-learn.org/
- Python Official Documentation
  https://docs.python.org/3/
- Pandas Documentation
  https://pandas.pydata.org/
-
- Jurafsky & Martin – Speech and Language Processing
-
- Online tutorials and learning resources related to:
  - Machine Learning
  - Natural Language Processing
  - Emotion Detection